

COMPSCI 589 - Final Project - Spring 2026

Due May 8, 11:55 pm Eastern Time

1 Datasets

1.1 The Hand-Written Digits Recognition Dataset

The goal here is to analyze 8×8 pictures of hand-written digits and classify them as belonging to one of 10 possible classes. Each class is associated with one particular digit: $0, 1, \dots, 9$. In this dataset, each instance consists of $8 \times 8 = 64$ numerical attributes. Each attribute represents the grayscale value of a single pixel in the image being classified. This dataset contains 1797 instances.

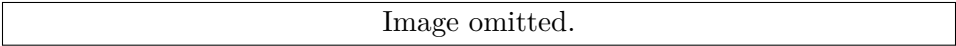


Figure 1: Examples of hand-written digits that you will be classifying.

To access this dataset, and for sample code describing how to load and pre-process it, please visit Scikit-learn's website. Further, below is a simple example of how to load this dataset; here, the instances are saved in `digits_dataset_X` and their corresponding classes are saved in `digits_dataset_y`.

Listing 1: Example showing how to load the Digits dataset from Scikit-learn and how to display information about a random digit from the dataset.

```
1 from sklearn import datasets
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 digits = datasets.load_digits(return_X_y=True)
6 digits_dataset_X = digits[0]
7 digits_dataset_y = digits[1]
8 N = len(digits_dataset_X)
9
10 # Prints the 64 attributes of a random digit, its class,
11 # and then shows the digit on the screen
12 digit_to_show = np.random.choice(range(N), 1)[0]
13 print("Attributes:", digits_dataset_X[digit_to_show])
14 print("Class:", digits_dataset_y[digit_to_show])
15
16 plt.imshow(np.reshape(digits_dataset_X[digit_to_show], (8, 8)))
17 plt.show()
```

Result:

Chosen Algorithm (3):

- KNN
- Random Forest
- Backpropagation Algorithm

Reason choose these algorithms:

In this dataset, each sample contains $8 \times 8 = 64$ numerical features. KNN was chosen because images of the same digit typically have similar pixel patterns. Random Forest was selected because it is also capable of capturing relationships between pixels. The backpropagation algorithm was chosen because neural networks are well-suited for processing image-based numerical inputs and are suitable for classifying the ten digits.

Implementation: ZiChen algorithm (stratified 10-fold cross-validation)

1) KNN:

- Hyperparameters : K neighbors
- Best Hyperparameter: $k = 3$
- Best Accuracy: 0.988309
- Best F1-Score: 0.988315

k_neighbors	acc	prec	recall	f1
1	0.987821219518421	0.964160401002506	1	0.987814674345634
3	0.988309422087308	0.969210526315789	1	0.988315039082841
5	0.987744855322539	0.974473684210526	1	0.987769839305443
7	0.986612187221408	0.969736842105263	1	0.986589364044086
9	0.984954593728254	0.954473684210526	1	0.984896288449694
11	0.984942489002256	0.953947368421053	1	0.984957686186014

Figure 2: KNN Hyperparameters for MNIST dataset.

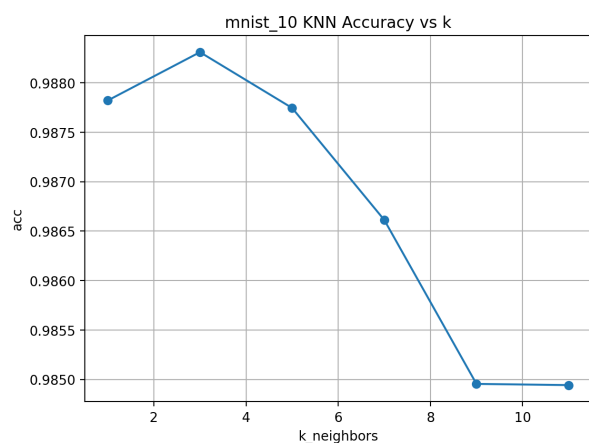


Figure 3: KNN Accuracy for MNIST dataset.

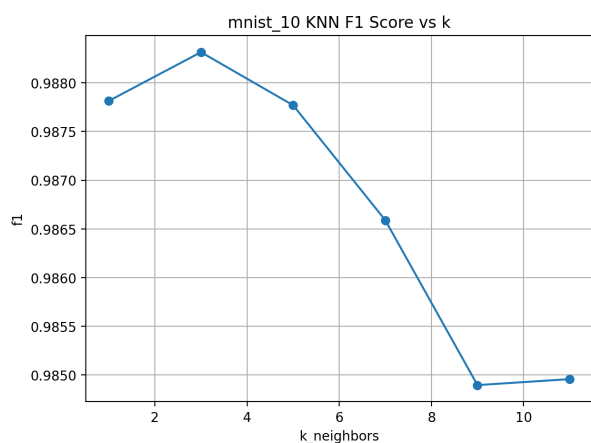


Figure 4: KNN F1-Score for MNIST dataset.

2) Random Forests:

- Hyperparameters: ntree, max depths, min splits
- Best Hyperparameter: ntree = 50, max depth = 8, min split = 2
- Best Accuracy: 0.9717
- Best F1-Score: 0.9716

ntrees	max_lenght	min_samples_split	acc	f1
1	8	2	0.745625222147138	0.744932425906181
5	8	2	0.894841918033802	0.894823159907037
10	8	2	0.936616032608275	0.936259640789199
20	8	2	0.953830355826084	0.953715152885182
30	8	2	0.964997180199979	0.964763009392791
40	8	2	0.967821026660468	0.967622201753531
50	8	2	0.971689097882764	0.971550885911567

Figure 5: Random Forest Hyperparameters for MNIST dataset.

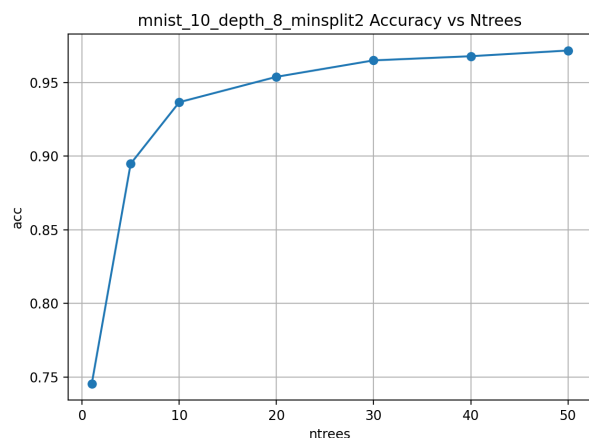


Figure 6: Random Forests Accuracy for MNIST dataset.

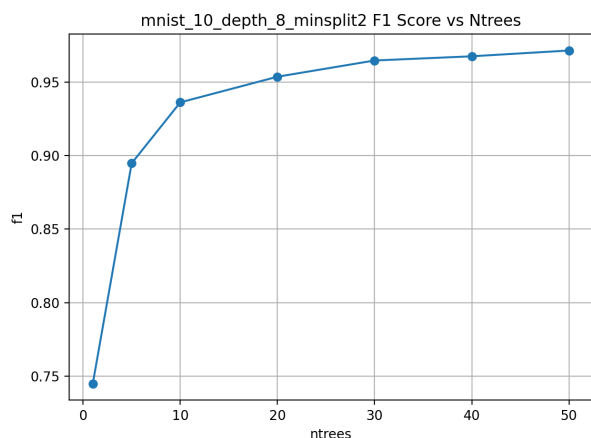


Figure 7: Random Forests F1-Score for MNIST dataset.

3) Backpropagation Algorithm:

- Hyperparameters: Architecture, λ
- Best Hyperparameter: Architecture = [64, 32, 10], $\lambda = 0.30$
- Best Accuracy: 0.9739
- Best F1-Score: 0.9740

arch	lambda	acc	f1
[64, 12, 10]	0.01	0.962818193534427	0.962742117450322
[64, 12, 10]	0.05	0.966597630408712	0.96657017863829
[64, 12, 10]	0.1	0.968841472440736	0.968986324334631
[64, 12, 10]	0.3	0.963872057420029	0.9639741408326
[64, 16, 10]	0.01	0.971150708898717	0.971156184471587
[64, 16, 10]	0.05	0.972213573847995	0.97211502627176
[64, 16, 10]	0.1	0.969968665789194	0.969991984309984
[64, 16, 10]	0.3	0.971101579220278	0.971094538968515
[64, 32, 10]	0.01	0.971638744845915	0.971755842287576
[64, 32, 10]	0.05	0.970506572247393	0.970558924003971
[64, 32, 10]	0.1	0.971596460440847	0.971676729667055
[64, 32, 10]	0.3	0.973898739451568	0.973986085945451
[64, 64, 10]	0.01	0.972759467299742	0.972803202944844
[64, 64, 10]	0.05	0.973318266021122	0.973389821005146
[64, 64, 10]	0.1	0.971096046741515	0.971233988617115
[64, 64, 10]	0.3	0.972200367027055	0.972368611886551
[64, 32, 16, 10]	0.01	0.969491834078739	0.969606968269507
[64, 32, 16, 10]	0.05	0.965580914032902	0.965638161153641
[64, 32, 16, 10]	0.1	0.963319954471194	0.963292888746449
[64, 32, 16, 10]	0.3	0.972193793584264	0.972237972635556
[64, 64, 32, 16, 10]	0.01	0.954497968738164	0.954419665524389
[64, 64, 32, 16, 10]	0.05	0.957678498782629	0.957880456001723
[64, 64, 32, 16, 10]	0.1	0.959313354581301	0.959325847888154
[64, 64, 32, 16, 10]	0.3	0.954887572010041	0.954781380187836

Figure 8: Backpropagation Hyperparameters for MNIST dataset.

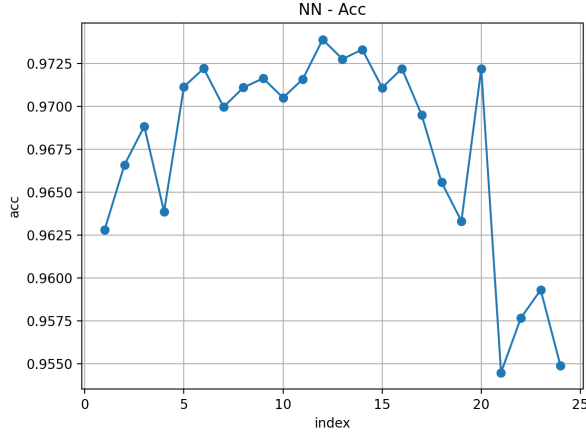


Figure 9: Backpropagation Accuracy for MNIST dataset.

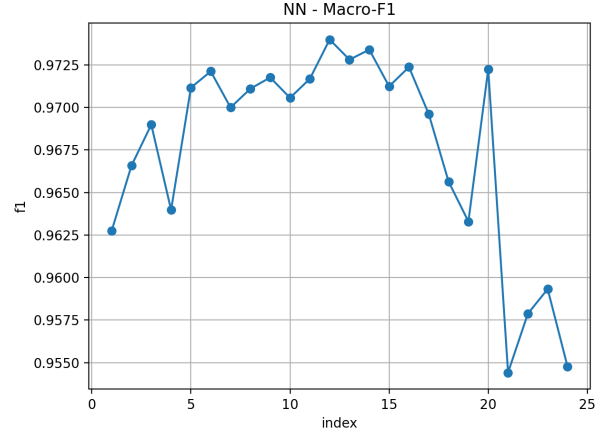


Figure 10: Backpropagation F1-Score for MNIST dataset.

Interpret Graph: On the Digits dataset, KNN performed best among the algorithms. The KNN curve shows that smaller values of k perform better, with $k = 3$ yielding the best results. As k increases, both accuracy and F1 score decline slightly, indicating that using too many neighbors can over-smooth the decision boundary. Random Forests also performed strongly, with performance increasing as the number of trees increased, however, after reaching a certain threshold, the improvement became negligible. Backpropagation performed well, however, its performance is more dependent on architecture and training parameters, so it performed slightly worse than KNN on this dataset.

1.2 The Oxford Parkinson's Disease Detection Dataset

This dataset is composed of a range of biomedical voice measurements from 31 people, 23 of whom are patients with Parkinson's disease. Each row in the dataset corresponds to the voice recording of one of these individuals. Each attribute corresponds to the measurement of a specific property of the patient's voice—for example, their average vocal frequency or several measures of frequency variation. The goal here is to predict whether a particular person is healthy or has Parkinson's. The binary target class to be predicted is Diagnosis: whether the patient is healthy, class 0, or a patient with Parkinson's, class 1. There are 195 instances in this dataset. All 22 attributes are numerical. The Parkinson's dataset can be found in this assignment's zip file on Canvas.

Result:

Chosen Algorithm (3):

- KNN
- Random Forest
- Backpropagation Algorithm

Reason choose these algorithms:

This dataset contains only numerical features. KNN was chosen because patients with similar voice measurements may have similar diagnostic labels. Random Forest was selected because it is also capable of capturing relationships among voice features. The backpropagation algorithm was used to evaluate whether the neural network could learn a better decision boundary from the 22 numerical attributes, although this dataset is small and prone to overfitting.

Implementation: ZiChen algorithm (stratified 10-fold cross-validation)

1) KNN:

- Hyperparameters : K neighbors
- Best Hyperparameter: $k = 1$
- Best Accuracy: 0.9595
- Best F1-Score: 0.9728

k_neighbors	acc	prec	recall	f1
1	0.9594444444444445	0.9808333333333333	0.9666666666666667	0.972834895916097
3	0.943918128654971	0.96125	0.9666666666666667	0.962981090100111
5	0.928918128654971	0.953214285714286	0.9533333333333333	0.952413793103448
7	0.913362573099415	0.925357142857143	0.9666666666666667	0.943997707890923
9	0.89780701754386	0.902979691876751	0.972857142857143	0.934884753167568
11	0.893099415204678	0.90327509951349	0.965714285714286	0.931764627121396

Figure 11: KNN Hyperparameters for Parkinson dataset.

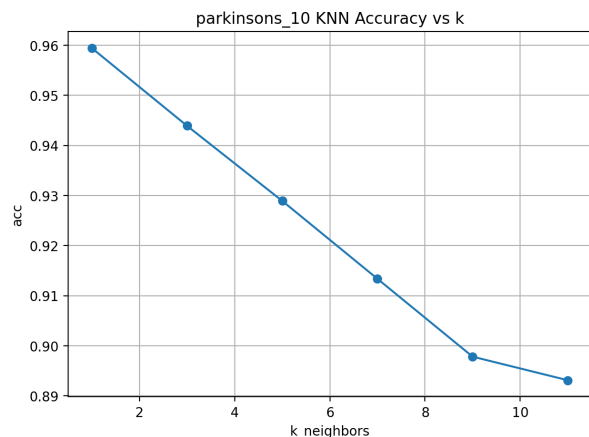


Figure 12: KNN Accuracy for Parkinson dataset.

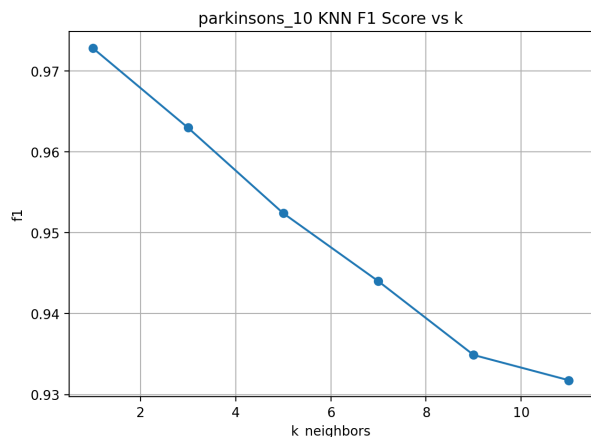


Figure 13: KNN F1-Score for Parkinson dataset.

2) Random Forests:

- Hyperparameters: ntree, max depths, min splits
- Best Hyperparameter: ntree = 50, max depth = 10, min split = 2
- Best Accuracy: 0.9334
- Best F1-Score: 0.9574

ntrees	acc	prec	recall	f1
1	0.770350877192983	0.845031512605042	0.857142857142857	0.849748896820643
5	0.872280701754386	0.894672304819364	0.944761904761905	0.917690906763487
10	0.888099415204678	0.907620214752568	0.952857142857143	0.928223994674217
20	0.897543859649123	0.901789215686275	0.972380952380953	0.934910295951731
30	0.913099415204678	0.912720588235294	0.97952380952381	0.944467000370782
40	0.918362573099415	0.913872549019608	0.986666666666667	0.948131720430108
50	0.933362573099415	0.931151960784314	0.986666666666667	0.957405913978495

Figure 14: Random Forest Hyperparameters for Parkinson dataset.

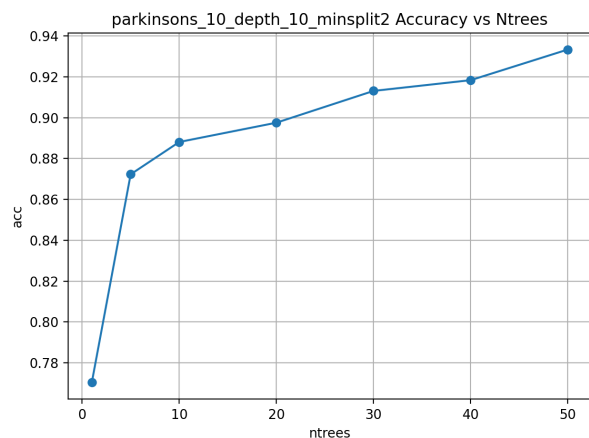


Figure 15: Random Forests Accuracy for Parkinson dataset.

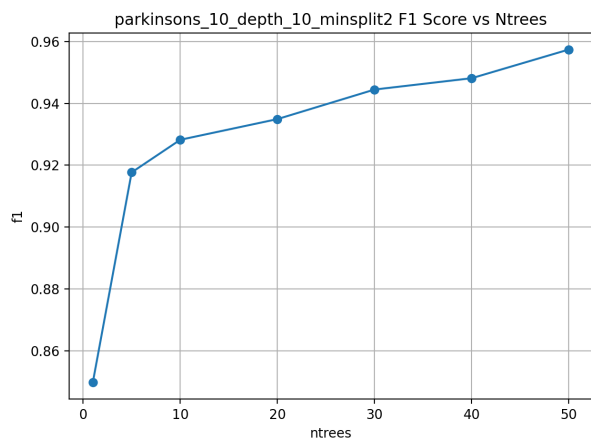


Figure 16: Random Forests F1-Score for Parkinson dataset.

3) Backpropagation Algorithm:

- Hyperparameters: Architecture, λ
- Best Hyperparameter: Architecture = [22, 8, 4, 1], $\lambda = 0.3$
- Best Accuracy: 0.8511
- Best F1-Score: 0.9055

arch	lambda	acc	f1
[22, 2, 1]	0.01	0.830877192982456	0.891559599368112
[22, 2, 1]	0.05	0.841140350877193	0.898436685018835
[22, 2, 1]	0.1	0.841140350877193	0.898436685018835
[22, 2, 1]	0.3	0.850877192982456	0.905501689906421
[22, 4, 1]	0.01	0.846140350877193	0.901845775927926
[22, 4, 1]	0.05	0.841140350877193	0.898436685018835
[22, 4, 1]	0.1	0.846140350877193	0.90209259899733
[22, 4, 1]	0.3	0.845877192982456	0.901936627517829
[22, 8, 1]	0.01	0.841140350877193	0.898436685018835
[22, 8, 1]	0.05	0.841140350877193	0.898280713539334
[22, 8, 1]	0.1	0.841140350877193	0.898280713539334
[22, 8, 1]	0.3	0.845877192982456	0.901936627517829
[22, 16, 1]	0.01	0.836140350877193	0.894871622630243
[22, 16, 1]	0.05	0.836140350877193	0.894871622630243
[22, 16, 1]	0.1	0.841140350877193	0.898280713539334
[22, 16, 1]	0.3	0.845877192982456	0.901936627517829
[22, 8, 4, 1]	0.01	0.85140350877193	0.90470615109225
[22, 8, 4, 1]	0.05	0.85140350877193	0.905310989801928
[22, 8, 4, 1]	0.1	0.841140350877193	0.898436685018835
[22, 8, 4, 1]	0.3	0.851140350877193	0.905523936847466
[22, 16, 8, 4, 1]	0.01	0.85140350877193	0.905310989801928
[22, 16, 8, 4, 1]	0.05	0.841140350877193	0.898436685018835
[22, 16, 8, 4, 1]	0.1	0.831140350877193	0.894971988795518
[22, 16, 8, 4, 1]	0.3	0.793918128654971	0.876416793990323
[22, 16, 8, 4, 2, 1]	0.01	0.802251461988304	0.885507703081233
[22, 16, 8, 4, 2, 1]	0.05	0.769239766081871	0.866991341991342
[22, 16, 8, 4, 2, 1]	0.1	0.754239766081871	0.859848484848485
[22, 16, 8, 4, 2, 1]	0.3	0.754239766081871	0.859848484848485

Figure 17: Backpropagation Hyperparameters for Parkinson dataset.

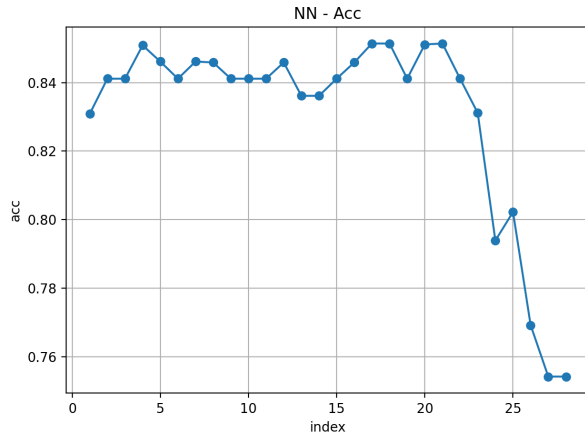


Figure 18: Backpropagation Accuracy for Parkinson dataset.

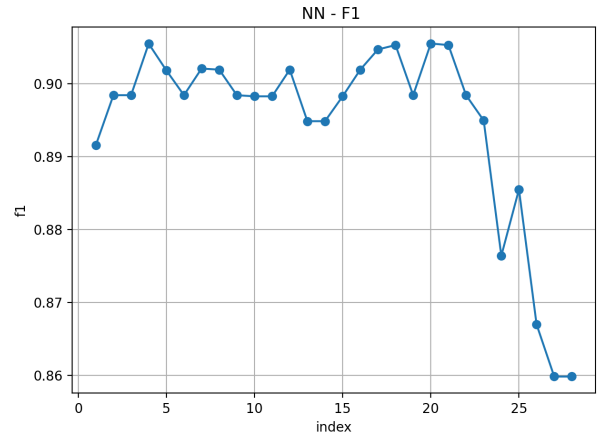


Figure 19: Backpropagation F1-Score for Parkinson dataset.

Interpret Graph: On the Parkinson disease dataset, KNN performed best among the algorithms tested. The KNN curve shows that smaller values of k perform better, with $k = 1$ yielding the best results. This suggests that using too many neighbors can over-smooth the decision boundary. Random Forests also performed strongly; results generally improved as the number of trees increased, but the improvement became negligible after reaching a certain threshold. Backpropagation performed well; however, its performance is more dependent on architecture and training parameters, so it performed slightly worse than KNN on this dataset.

1.3 The Rice Grains Dataset

This dataset contains information about 3810 rice grain images. In particular, each rice grain instance is described by 7 numerical morphological features inferred from images of different rice grains, e.g., the longest line that can be drawn on the rice grain. The goal is to predict whether a given rice grain belongs to the Cammeo species or the Osmancik species. This dataset can be found in this assignment's zip file on Canvas.

Result:

Chosen Algorithm (3):

- KNN
- Random Forest
- Backpropagation Algorithm

Reason choose these algorithms:

This dataset contains numerical features. KNN was chosen because the shape measurements of rice grains of the same variety are expected to be similar. Random Forest was selected because it can capture the relationships among the seven morphological features. The backpropagation algorithm was included to test whether the neural network can learn a more complex decision boundary between the Cammeo and Osmancik categories. This dataset contains only numerical features.

Implementation: ZiChen algorithm (stratified 10-fold cross-validation)

1) KNN:

- Hyperparameters : K neighbors
- Best Hyperparameter: $k = 11$
- Best Accuracy: 0.9228
- Best F1-Score: 0.9094

k_neighbors	acc	prec	recall	f1
1	0.88503937007874	0.8611111655501597	0.87239263803681	0.866514161826395
3	0.909448818897638	0.895494654055546	0.893251533742331	0.893975090695445
5	0.917322834645669	0.905102334431568	0.901840490797546	0.903172582055348
7	0.91994750656168	0.908529643196665	0.904294478527607	0.906141685627985
9	0.921259842519685	0.910795118886856	0.904907975460123	0.907604334921922
11	0.922834645669292	0.91224308353132	0.907361963190184	0.90943592040154

Figure 20: KNN Hyperparameters for Rice dataset.

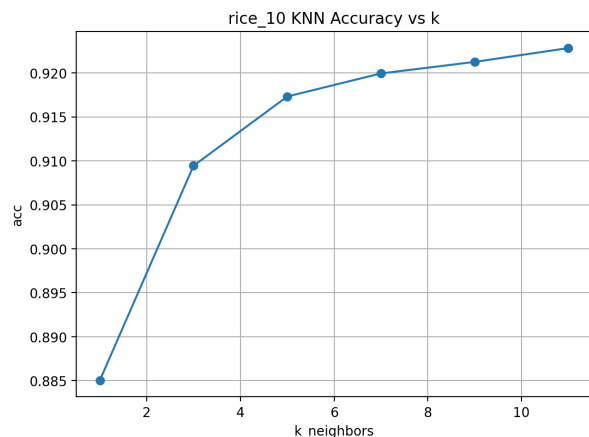


Figure 21: KNN Accuracy for Rice dataset.

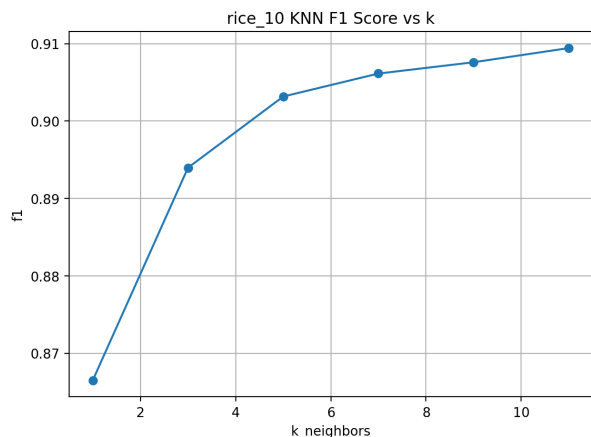


Figure 22: KNN F1-Score for Rice dataset.

2) Random Forests:

- Hyperparameters: ntree, max depths, min splits
- Best Hyperparameter: ntree = 50, max depth = 8, min split = 2
- Best Accuracy: 0.9276
- Best F1-Score: 0.9148

ntrees	acc	prec	recall	f1
1	0.918635170603675	0.909935659391331	0.899386503067485	0.904331419552617
5	0.923359580052493	0.919662088503091	0.9	0.909287805018587
10	0.925984251968504	0.915472421784158	0.911656441717791	0.913284669853439
20	0.927296587926509	0.918705574159929	0.911042944785276	0.91456764871964
30	0.925984251968504	0.916908637502276	0.909815950920246	0.913043987990052
40	0.926509186351706	0.91802047504316	0.909815950920246	0.913618719098454
50	0.92755905511811	0.91918286043705	0.911042944785276	0.914849616509134

Figure 23: Random Forests Hyperparameters for Rice dataset.

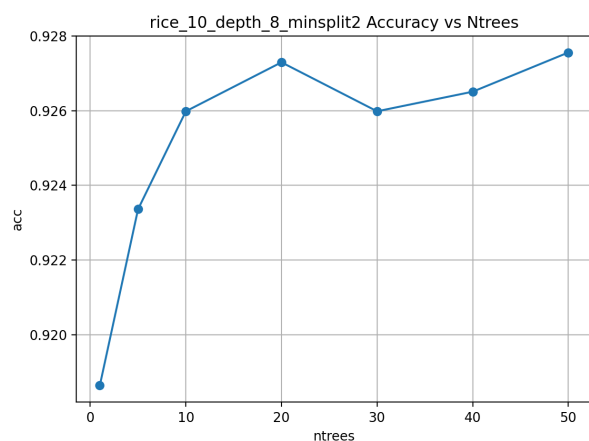


Figure 24: Random Forests Accuracy for Rice dataset.

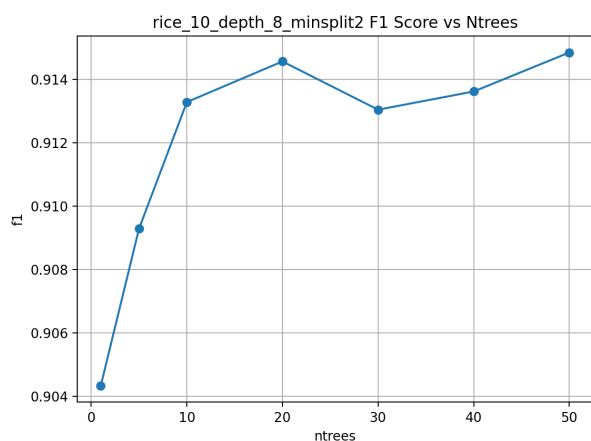


Figure 25: Random Forests F1-Score for Rice dataset.

3) Backpropagation Algorithm:

- Hyperparameters: Architecture, λ
- Best Hyperparameter: Architecture = [7, 16, 1], $\lambda = 0.3$
- Best Accuracy: 0.9286
- Best F1-Score: 0.9166

arch	lambda	acc	f1
[7, 2, 1]	0.01	0.927821522309711	0.915554500015581
[7, 2, 1]	0.05	0.927296587926509	0.914861581465949
[7, 2, 1]	0.1	0.927296587926509	0.914895293218981
[7, 2, 1]	0.3	0.928346456692913	0.916136591212646
[7, 4, 1]	0.01	0.927821522309711	0.915661041393318
[7, 4, 1]	0.05	0.927034120734908	0.914592139503529
[7, 4, 1]	0.1	0.928083989501312	0.915942673869207
[7, 4, 1]	0.3	0.927296587926509	0.91492631872902
[7, 8, 1]	0.01	0.927821522309711	0.915611141193717
[7, 8, 1]	0.05	0.92755905511811	0.915338090990369
[7, 8, 1]	0.1	0.927821522309711	0.915600992577809
[7, 8, 1]	0.3	0.927296587926509	0.914982101991689
[7, 16, 1]	0.01	0.92755905511811	0.915369326641145
[7, 16, 1]	0.05	0.928346456692913	0.916281289608066
[7, 16, 1]	0.1	0.928083989501312	0.915992982493266
[7, 16, 1]	0.3	0.928608923884515	0.916550123605705
[7, 8, 4, 1]	0.01	0.927296587926509	0.91483358640779
[7, 8, 4, 1]	0.05	0.927296587926509	0.914918530567026
[7, 8, 4, 1]	0.1	0.927296587926509	0.914822045598153
[7, 8, 4, 1]	0.3	0.927034120734908	0.914551454746348
[7, 16, 8, 4, 1]	0.01	0.926771653543307	0.914269073413826
[7, 16, 8, 4, 1]	0.05	0.926771653543307	0.914298599488033
[7, 16, 8, 4, 1]	0.1	0.92755905511811	0.915168477373522
[7, 16, 8, 4, 1]	0.3	0.927296587926509	0.91488634571208
[7, 16, 8, 4, 2, 1]	0.01	0.681627296587926	0.27782207537081
[7, 16, 8, 4, 2, 1]	0.05	0.715748031496063	0.367565389736734
[7, 16, 8, 4, 2, 1]	0.1	0.853805774278215	0.728706544831374
[7, 16, 8, 4, 2, 1]	0.3	0.712073490813648	0.362936981019447

Figure 26: Backpropagation Hyperparameters for Rice dataset.

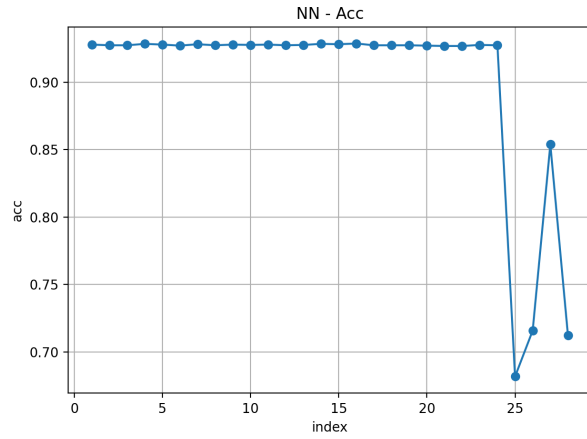


Figure 27: Backpropagation Accuracy for Rice dataset.

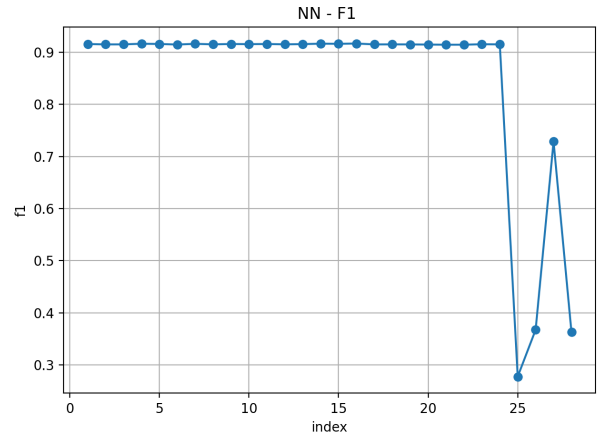


Figure 28: Backpropagation F1-Score for Rice dataset.

Interpret Graph: For the rice dataset, the backpropagation algorithm achieved the highest accuracy and F1 score, while the random forest algorithm performed very similarly. This dataset contains 3,810 samples and only 7 numerical morphological features, making it highly suitable for neural networks. The performance curve for the Random Forest algorithm indicates that increasing the number of trees initially improves performance, but when the ensemble size becomes sufficiently large, performance actually decreases. The backpropagation curve shows that the neural network is highly capable of learning useful nonlinear relationships between morphological features and rice species. KNN also performed quite well, though slightly worse than the other two algorithms.

1.4 The Credit Approval Dataset

This dataset contains information about 653 credit card applications. The attribute names and their values were converted from their original form to a format that is not easily interpretable by humans, in order to protect the confidentiality of the data. That said, they still contain all the necessary information to train effective classifiers. The goal is to predict whether a given credit card application will be approved. Each credit card application is described by 6 numerical features and 9 categorical features. This dataset can be found in this assignment's zip file on Canvas.

Result:

Chosen Algorithm (3):

- Decision Tree
- Random Forest
- Backpropagation Algorithm

Reason choose these algorithms

This dataset contains both numerical and categorical features, and the task is a binary classification problem. Decision trees were chosen because they are well-suited for tabular data and can learn effective decision rules after converting categorical features using one-hot encoding. Random Forests were chosen because they improve decision stability and reduce overfitting by combining multiple decision trees. Backpropagation was used to evaluate whether a neural network could learn a better decision boundary from the encoded features, although the dataset is relatively small and prone to overfitting.

Implementation: ZiChen algorithm (stratified 10-fold cross-validation)

1) Decision Tree:

- Hyperparameters : min split, max lenght
- Best Hyperparameter: min split = 5, max lenght = 3
- Best Accuracy: 0.8668
- Best F1-Score: 0.8663

Min samples split	Max lenght	acc	prec	recall	f1
5	2	0.865200320512821	0.800934852580963	0.942298850574713	0.8643049965222
5	3	0.866762820512821	0.801427463418401	0.945747126436782	0.866301366758135
5	5	0.860773601398601	0.817322580361905	0.89551724137931	0.853720447883882
5	8	0.848487033799534	0.832376318604767	0.841609195402299	0.834354034867786
5	10	0.833098776223776	0.815937949379134	0.820919540229885	0.816636364881865
5	15	0.82235067016317	0.798305216520595	0.817701149425287	0.806677614031026

Figure 29: Decision Tree Hyperparameters for Credit dataset.

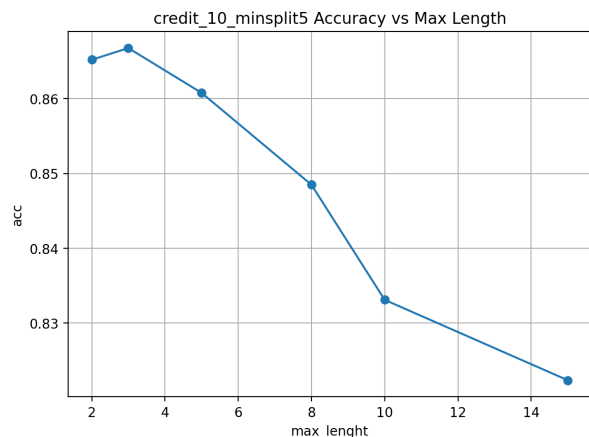


Figure 30: Decision Tree Accuracy for Credit dataset.

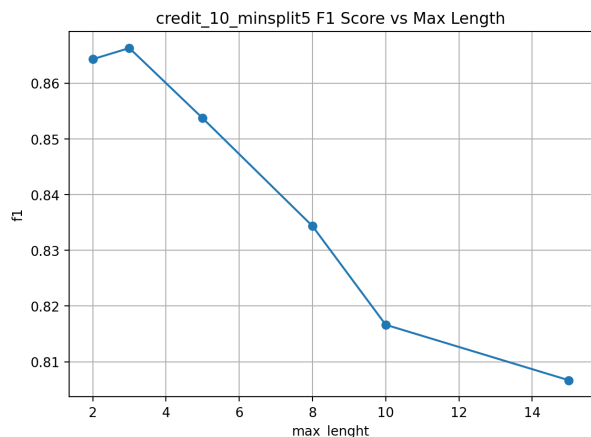


Figure 31: Decision Tree F1-Score for Credit dataset.

2) Random Forests:

- Hyperparameters: ntree, max depths, min splits
- Best Hyperparameter: ntree = 30, max depth = 5, min split = 2
- Best Accuracy: 0.8728
- Best F1-Score: 0.8666

ntrees	acc	prec	recall	f1
1	0.859116404428905	0.805204599322246	0.915287356321839	0.855477876537932
5	0.86958041958042	0.826532182195133	0.905057471264368	0.862915575750246
10	0.871332313519813	0.837657053046047	0.891954022988506	0.862558368111629
20	0.871237616550117	0.826755952380953	0.908620689655172	0.865064867095294
30	0.872847465034965	0.830464104996837	0.908735632183908	0.866567326857708
40	0.871284965034965	0.826162464985994	0.912068965517241	0.865633479651386
50	0.869769813519813	0.825626284707167	0.908620689655172	0.863932398073679

Figure 32: Random Forest Hyperparameters for Credit dataset.

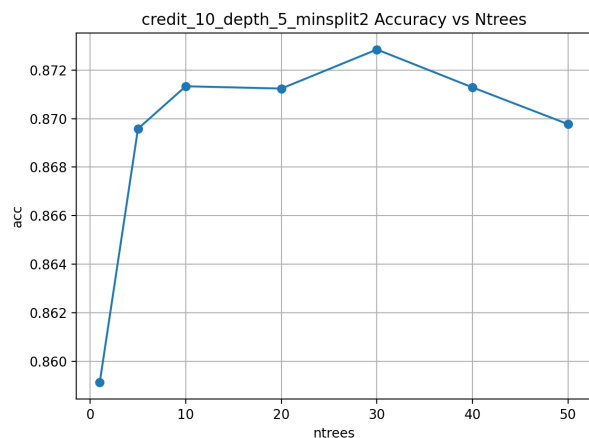


Figure 33: Random Forests Accuracy for Credit dataset.

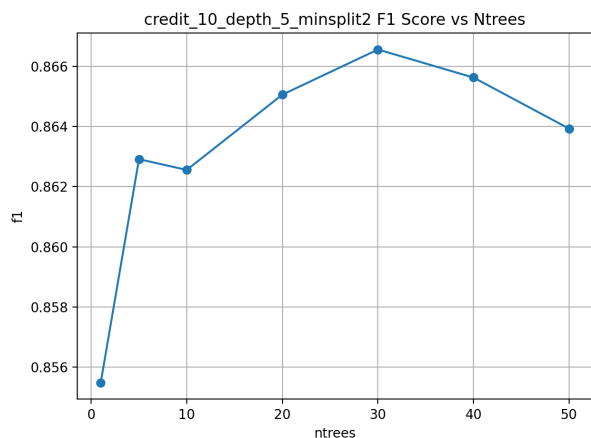


Figure 34: Random Forests F1-Score for Credit dataset.

3) Backpropagation Algorithm:

- Hyperparameters: Architecture, λ
- Best Hyperparameter: Architecture = [68, 16, 1], $\lambda = 0.3$
- Best Accuracy: 0.8623
- Best F1-Score: 0.8514

arch	lambda	acc	f1
[68, 2, 1]	0.01	0.847043269230769	0.83162402788788
[68, 2, 1]	0.05	0.853103875291375	0.838591432321041
[68, 2, 1]	0.1	0.854689685314685	0.840675161651422
[68, 2, 1]	0.3	0.851469988344988	0.836043819997461
[68, 4, 1]	0.01	0.848534382284382	0.832963063332622
[68, 4, 1]	0.05	0.848463723776224	0.832132354298604
[68, 4, 1]	0.1	0.848416375291375	0.834030761621368
[68, 4, 1]	0.3	0.856180798368298	0.842735784093257
[68, 8, 1]	0.01	0.859211101398601	0.845560493406638
[68, 8, 1]	0.05	0.859305798368298	0.846631412443659
[68, 8, 1]	0.1	0.851564685314685	0.835297545700843
[68, 8, 1]	0.3	0.854713723776224	0.840593364930434
[68, 16, 1]	0.01	0.856157488344988	0.842578496374814
[68, 16, 1]	0.05	0.848534382284382	0.833993140992162
[68, 16, 1]	0.1	0.857719988344988	0.844223700591755
[68, 16, 1]	0.3	0.862289481351981	0.851352833338679
[68, 8, 4, 1]	0.01	0.850072843822844	0.832214007737722
[68, 8, 4, 1]	0.05	0.857838723776224	0.841204378944807
[68, 8, 4, 1]	0.1	0.851564685314685	0.835944790641931
[68, 8, 4, 1]	0.3	0.853079836829837	0.839058580131471
[68, 16, 8, 4, 1]	0.01	0.848629079254079	0.831338512292999
[68, 16, 8, 4, 1]	0.05	0.850049533799534	0.833606236073253
[68, 16, 8, 4, 1]	0.1	0.851612033799534	0.835163462672294
[68, 16, 8, 4, 1]	0.3	0.850049533799534	0.835324196957951
[68, 16, 8, 4, 2, 1]	0.01	0.824386655011655	0.755258494984756
[68, 16, 8, 4, 2, 1]	0.05	0.72337703962704	0.497662276615212
[68, 16, 8, 4, 2, 1]	0.1	0.791574155011655	0.670707957279355
[68, 16, 8, 4, 2, 1]	0.3	0.730479312354312	0.501967388258372

Figure 35: Backpropagation Hyperparameters for Credit dataset.

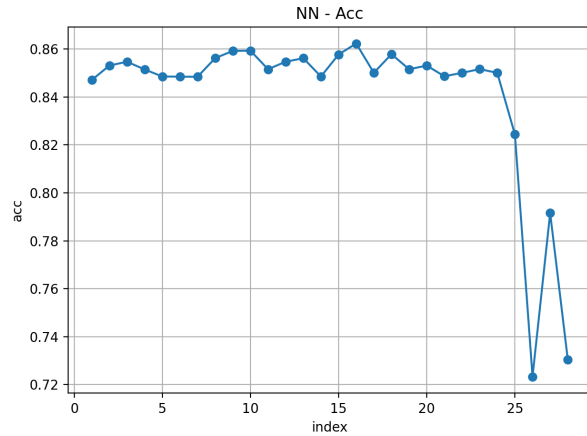


Figure 36: Backpropagation Accuracy for Credit dataset.

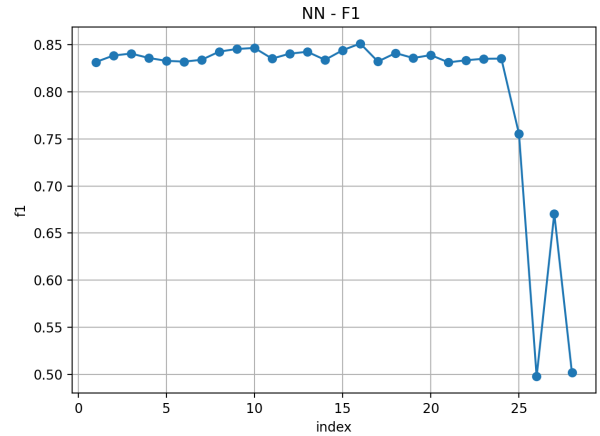


Figure 37: Backpropagation F1-Score for Credit dataset.

Interpret Graph: On the Credit dataset, Random Forests achieved the best overall performance. This dataset contains both numerical and categorical features; after one-hot encoding, the number of features increases. Decision Trees performed reasonably well, but a single tree is susceptible to subtle variations in the training data and may overfit to specific classifications. Random Forests mitigate overfitting by averaging the predictions of multiple trees. Backpropagation performed slightly worse, likely because the data became sparse after one-hot encoding. Techniques such as embeddings might alleviate this issue, but unfortunately, we are unable to use other ML libraries.

Overall Result:

Table 1: Overall best performance of each algorithm on each dataset. The gray cells indicate the highest value for each metric within the corresponding dataset.

Algorithm	Digits		Parkinson		Rice		Credit	
	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
KNN	0.9883	0.9883	0.9595	0.9728	0.9228	0.9094	–	–
Decision Tree	–	–	–	–	–	–	0.8668	0.8663
Random Forests	0.9717	0.9716	0.9334	0.9574	0.9276	0.9148	0.8728	0.8666
Backpropagation	0.9739	0.9740	0.8511	0.9055	0.9286	0.9166	0.8623	0.8514

2 Extra Credit

There are four ways in which we may receive extra credit on this assignment.

If you choose to work on any of the tasks below, please clearly indicate in your report, in red, which specific tasks you are working on for extra credit. Describe what you did as part of that task and present the corresponding results. Any source code you develop should be included in your Gradescope submission.

Extra Credit Question #1: 10 Points

You may earn up to 10 extra points if you analyze all four datasets using an additional algorithm. In other words, you would be evaluating more than two algorithms on each of the datasets discussed in Section 1.

- Digits: KNN + Random Forests + Backpropagation
- Credit: Decision Tree + Random Forests + Backpropagation
- Parkinson: KNN + Random Forest + Backpropagation
- Rice: KNN + Random Forest + Backpropagation

Extra Credit Question #2: 10 Points

You may earn up to 10 extra points if you select a new challenging dataset and evaluate the performance of different algorithms on it. A challenging dataset should necessarily include both categorical and numerical attributes and have more than two classes; or it should be a dataset where you have to classify images.

Dataset: Reels/Shorts Consumption vs Attention Span

<https://www.kaggle.com/datasets/jayjoshi37/reelssshorts-consumption-vs-attention-span>

We used "stress level" as the target variable, which contains three classes: Low, Medium, and High.

spam = span

Result:

Chosen Algorithm (2):

- KNN
- Backpropagation Algorithm

1) KNN:

- Hyperparameters : K
- Best Hyperparameter: K = 5
- Best Accuracy: 0.3361
- Best F1-Score: 0.3360

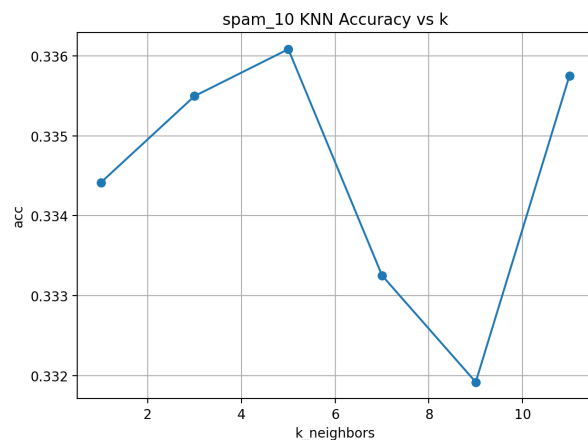


Figure 38: KNN Accuracy for Spam dataset.

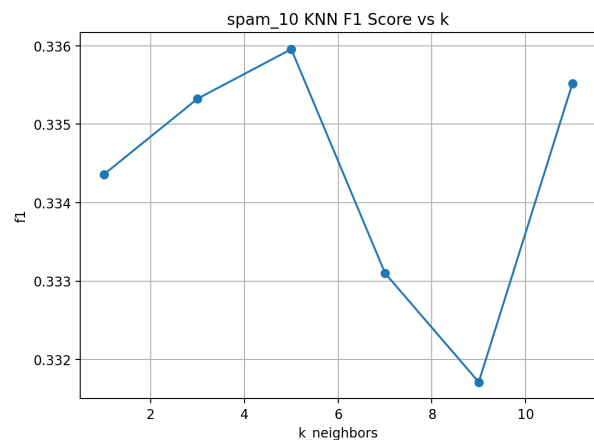


Figure 39: KNN F1-Score for Spam dataset.

2) Backpropagation Algorithm:

- Hyperparameters: Architecture, λ
- Best Hyperparameter: Architecture = [11, 8, 3], $\lambda = 0.01$
- Accuracy: 0.3386
- F1-Score: 0.3346

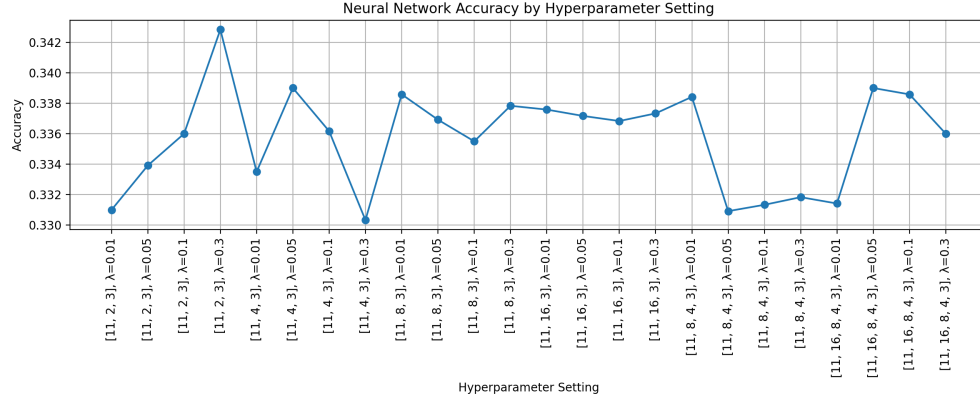


Figure 40: Backpropagation Accuracy for Spam dataset.

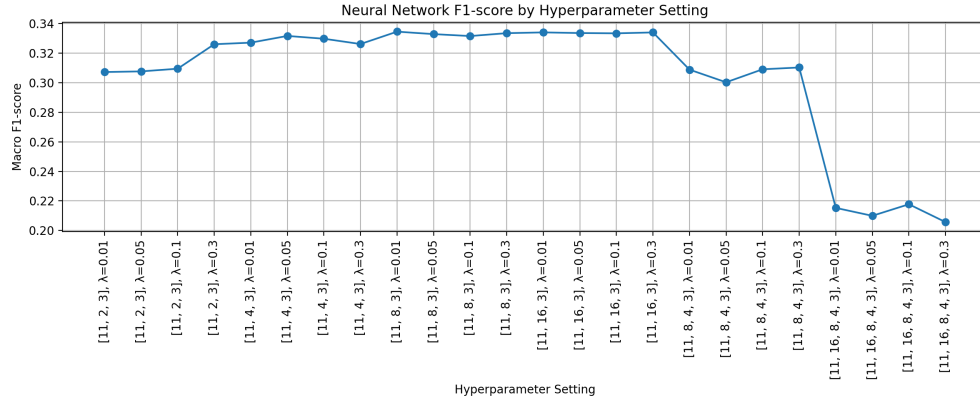


Figure 41: Backpropagation F1-Score for Spam dataset.

Interpret Graph: Both KNN and backpropagation achieved performance scores close to 0.33, suggesting that the features we selected may not effectively distinguish among the three target categories. KNN achieved the best F1 score, while the neural network achieved a slightly higher accuracy under certain hyperparameter settings. Overall, neither model demonstrated strong predictive performance on this dataset.

Extra Credit Question #4: 15 Points

We implemented a weighted KNN, where in our variant, closer neighbors receive greater voting weights. For a neighbor with Euclidean distance d , we compute its voting weight as: $\text{weight} = 1 / (\text{sqrt}(d) + \text{epsilon})$. For each class, we sum the weights of all neighbors belonging to that class and predict the class with the highest total weight.

Result:

Digits:

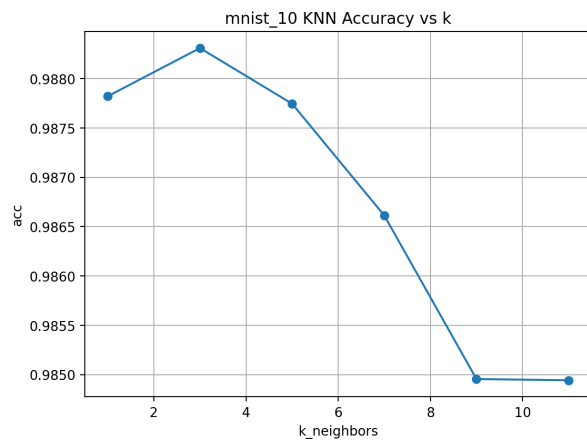


Figure 42: KNN Accuracy for MNIST dataset.

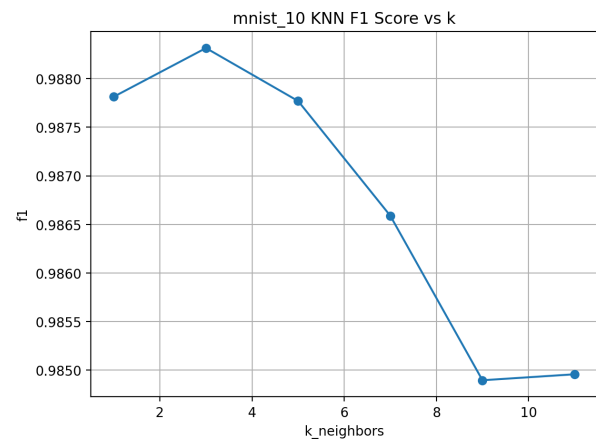


Figure 43: KNN F1-Score for MNIST dataset.

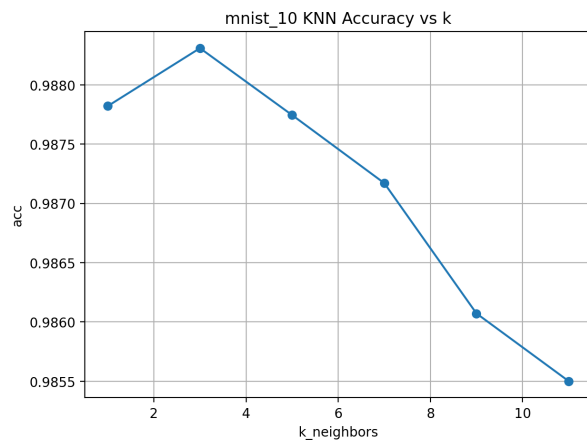


Figure 44: Weighted KNN Accuracy for MNIST dataset.

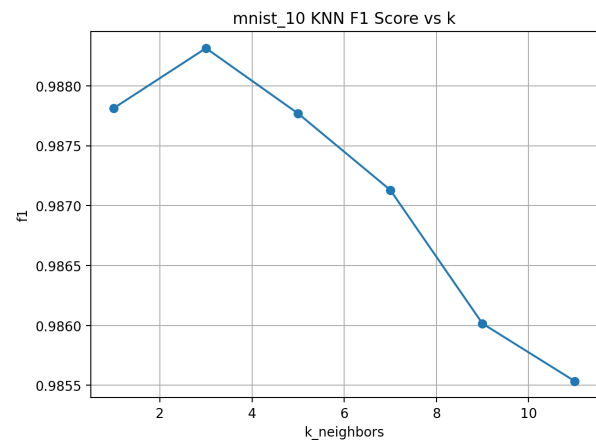


Figure 45: Weighted KNN F1-Score for MNIST dataset.

Parkinson:

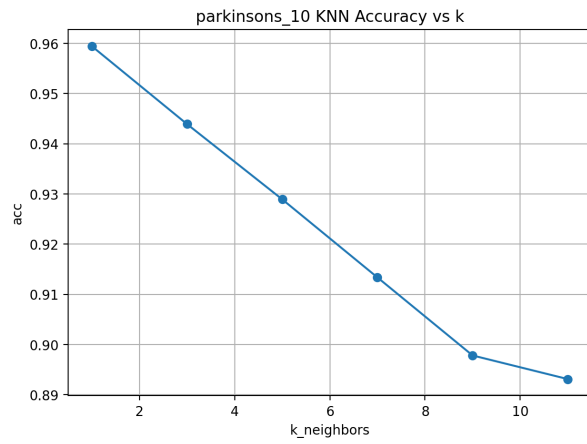


Figure 46: KNN Accuracy for Parkinson dataset.

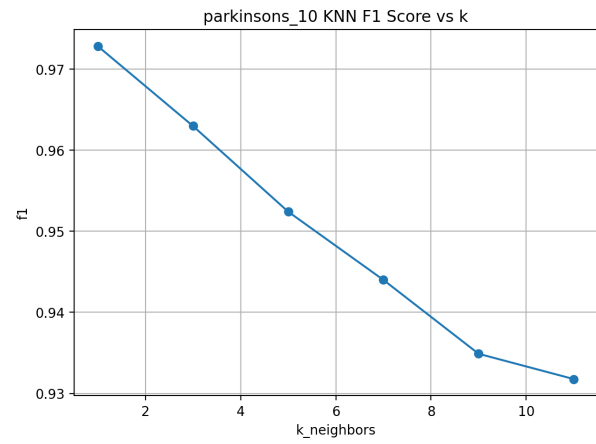


Figure 47: KNN F1-Score for Parkinson dataset.

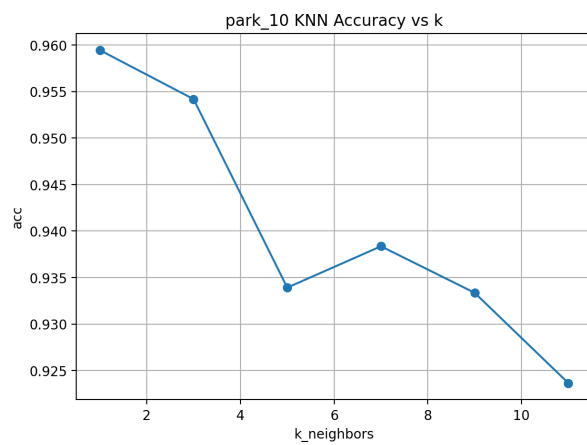


Figure 48: Weighted KNN Accuracy for Parkinson dataset.

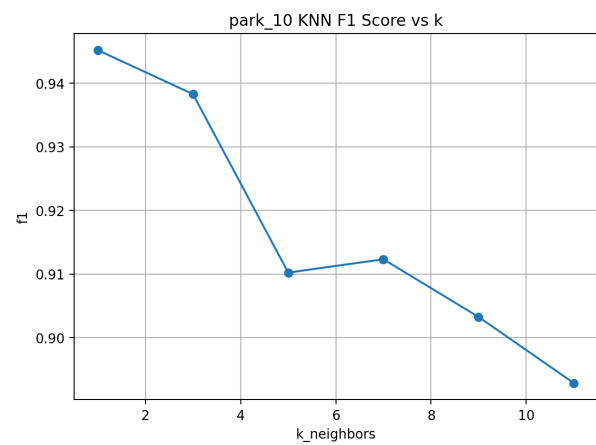


Figure 49: Weighted KNN F1-Score for Parkinson dataset.

Rice:

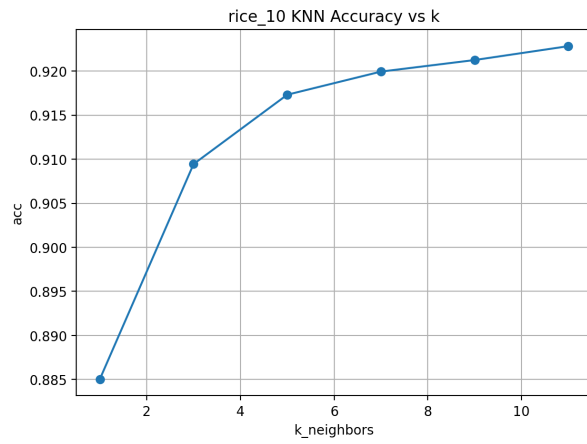


Figure 50: KNN Accuracy for Rice dataset.

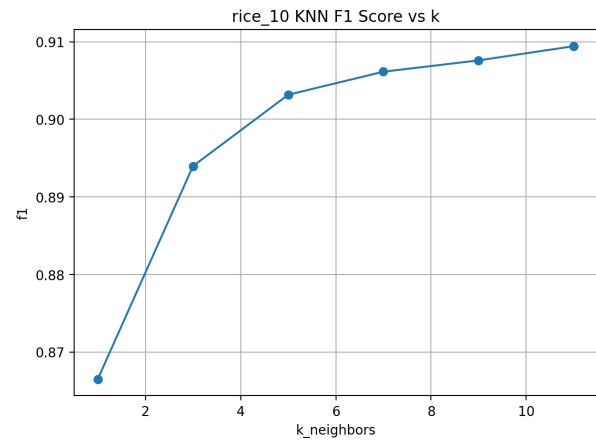


Figure 51: KNN F1-Score for Rice dataset.

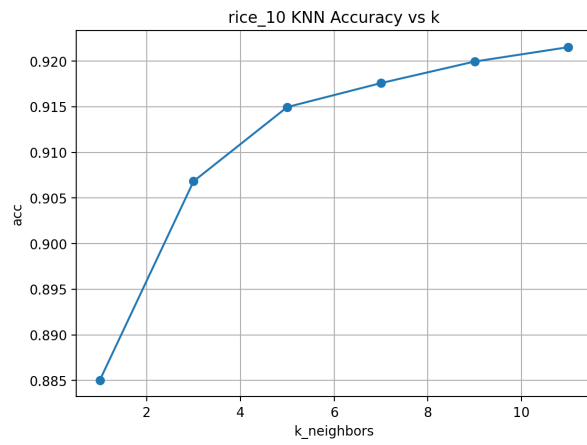


Figure 52: Weighted KNN Accuracy for Rice dataset.

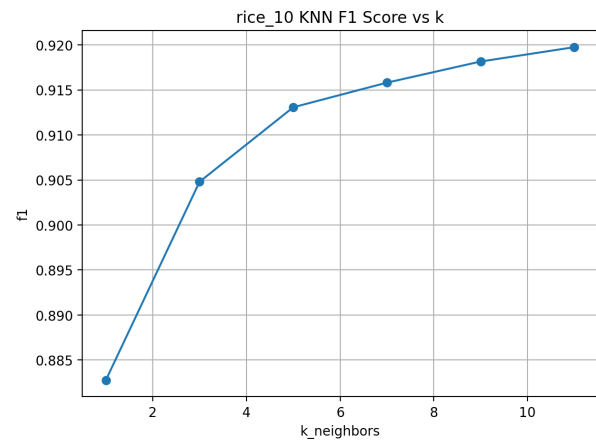


Figure 53: Weighted KNN F1-Score for Rice dataset.

Credit:

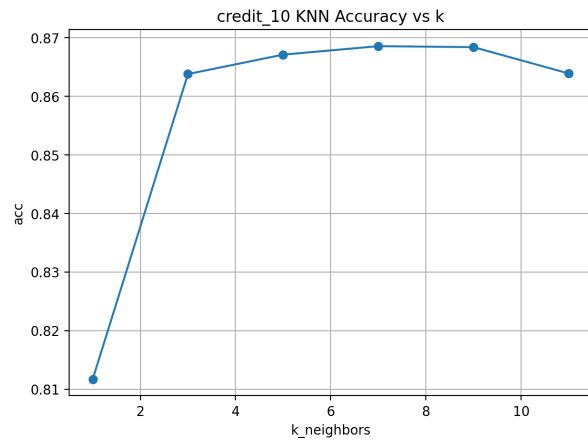


Figure 54: KNN Accuracy for Credit dataset.

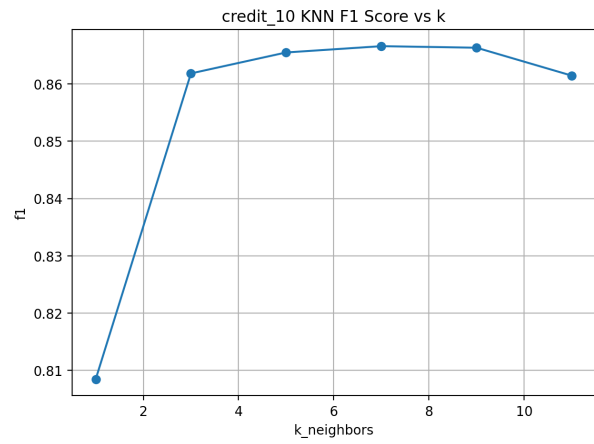


Figure 55: KNN F1-Score for Credit dataset.

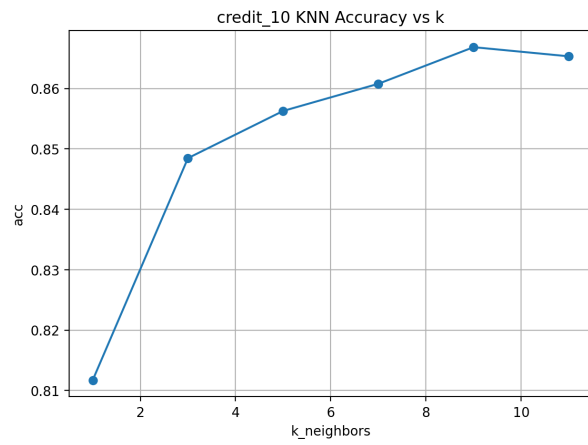


Figure 56: Weighted KNN Accuracy for Credit dataset.

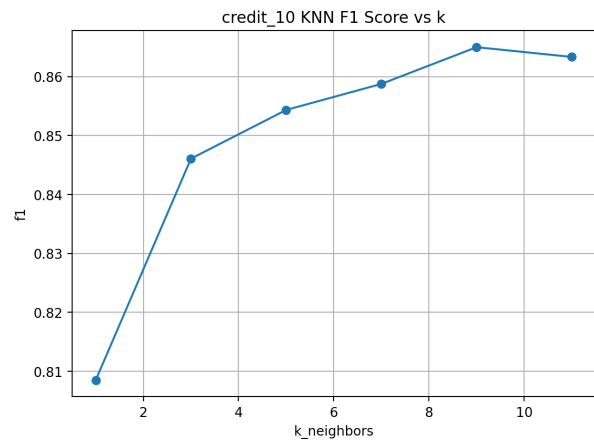


Figure 57: Weighted KNN F1-Score for Credit dataset.